

MDS-006 — Composition Engine

External publication draft

MDS-006 — Composition Engine

The interface is never authored. It is solved.

Purpose

Every specification preceding MDS-006 has defined **what** Mosaic is.

The MDL established:

- Vision
- Principles
- Mental Model
- Interaction
- Composition

The MDS established:

- Design Tokens
- Colour
- Materials
- Typography
- Motion

MDS-006 defines how all of those systems become a living runtime experience.

It is the architectural heart of Mosaic.

Unlike traditional UI frameworks, which render predefined screens, the Composition Engine continuously constructs the user's current World from behaviour, information and relationships.

It does not render layouts.

It solves understanding.

Relationship to Previous Specifications

[text]

Vision

↓

Principles

↓

Mental Model

↓

Interaction

↓

Composition

↓

Tokens

↓

Colour

↓

Materials

↓

Typography

↓

Motion

↓

Composition Engine

↓

Tile Framework

↓

Components

↓

Presentation

The Composition Engine consumes every previous specification.

It produces:

- Expressions
- Runtime Hierarchy
- Adaptive Composition
- Runtime Behaviour

- Presentation Models

Scope

This specification defines:

- Composition Engine Architecture
- Runtime Solver
- Expression Resolution
- Hierarchy Resolution
- Adaptive Layout Resolution
- Behaviour Orchestration
- Runtime Graph Processing
- Multi-Device Composition
- Composition Caching
- Runtime Pipelines

This specification intentionally does **not** define:

- Components
- Rendering APIs
- GraphQL Schema
- Storage
- Transport

Those systems provide data.

The Composition Engine constructs experience.

Core Question

MDS-006 exists to answer one question.

How does Mosaic construct the user's World at runtime?

Not:

How do we render screens?

Composition Engine Statement

Within Mosaic:

The user experiences a solved World rather than a rendered interface.

Every runtime decision should reinforce that principle.

Composition Engine Responsibilities

The Composition Engine separates runtime construction into several conceptual layers.

[text]

World

↓

Behaviour

↓

Composition Solver

↓

Expression Resolution

↓

Presentation Model

↓

Rendering

Each layer contributes one responsibility.

No layer duplicates another.

Expected Outcome

After reading MDS-006 contributors should understand:

- how runtime composition works,
- how Expressions are resolved,
- how adaptive composition behaves,
- how multiple devices share one runtime model,
- how behavioural changes propagate,
- how presentation remains implementation independent,

without discussing specific UI frameworks.

Repository Structure

[text]
design/

■■■ mds/

■■■ MDS-006 Composition Engine/

README.md

00-document-control.md

01-composition-engine-philosophy.md

02-runtime-world.md

03-composition-solver.md

04-expression-resolution.md

05-runtime-hierarchy.md

06-adaptive-layout.md

07-behaviour-orchestration.md

08-runtime-pipelines.md

09-composition-caching.md

10-multi-device-composition.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 → MDL-005
- MDS-001 → MDS-005

Downstream specifications:

- MDS-007 Tile Framework
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Runtime Architecture Team

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDS-006
Title	Mosaic Design System — Composition Engine
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Runtime Architecture Team
Parent Specifications	MDL-001 → MDL-005, MDS-001 → MDS-005
Repository	/design/mds/MDS-006 Composition Engine/

Purpose

MDS-006 defines the runtime architecture responsible for constructing the user's World.

Every previous specification described:

- what the platform believes,
- how it behaves,
- how it communicates.

The Composition Engine is responsible for turning those architectural concepts into a living experience.

Unlike conventional UI frameworks, which render predefined interface trees, the Composition Engine continuously solves:

- behavioural intent,
- hierarchy,
- expressions,
- presentation,

before a single component is rendered.

The Composition Engine is therefore the runtime embodiment of the Mosaic Design Language.

Authority

MDS-006 governs:

- Runtime World construction
- Composition Solver implementation

- Expression Resolution
- Runtime Hierarchy
- Behaviour Orchestration
- Adaptive Layout Resolution
- Composition Pipelines
- Runtime Caching
- Multi-device Composition

This specification intentionally does **not** govern:

- rendering frameworks,
- GraphQL schemas,
- storage,
- transport,
- platform widgets.

Those systems provide capabilities.

The Composition Engine creates experience.

Relationship To MDS

The Composition Engine consumes every conceptual system defined before it.

[mermaid]
flowchart TD

```
Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Colour
Colour --> Materials
Materials --> Typography
Typography --> Motion
Motion --> CompositionEngine
CompositionEngine --> Presentation
```

Everything before the Composition Engine defines intent.

Everything after it implements presentation.

Design Intent

Traditional applications typically follow:

[text]
Data

↓

Screen

↓

Render

Mosaic intentionally follows:

[text]
World

↓

Behaviour

↓

Composition

↓

Expressions

↓

Presentation

↓

Render

The distinction is fundamental.

Applications render components.

Mosaic constructs understanding.

Reader Expectations

Before reading this specification contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System

The Composition Engine assumes every conceptual decision has already been made.

Its responsibility is runtime orchestration.

Architectural Scope

The Composition Engine defines:

- runtime solving
- behavioural orchestration
- expression selection
- hierarchy resolution
- adaptive composition
- presentation modelling

It intentionally avoids implementation technologies such as:

- Flutter Widgets
- React Components
- SwiftUI Views
- Compose Composables

These become consumers of the Presentation Model produced by the engine.

Stability

Expected lifetime.

Artefact	Expected Lifetime
UI Components	Months
Rendering Backends	Months
Runtime Optimisations	Years
Composition Engine Architecture	Years
Runtime Philosophy	Decades

The engine implementation may evolve continuously.

Its conceptual model should remain remarkably stable.

Success Criteria

MDS-006 succeeds when:

- every client constructs identical understanding
- behaviour consistently produces identical composition
- adaptive layouts preserve the user's World
- extensions integrate naturally
- rendering frameworks remain replaceable

- contributors think in runtime worlds rather than interface trees

Users should never feel that screens are loading.

They should simply feel that their World continuously evolves around them.

Review Status

Status

Draft

Dependencies

- MDL-001 → MDL-005
- MDS-001 → MDS-005

Supersedes

None.

Next File

01-composition-engine-philosophy.md

Composition Engine Philosophy

Purpose

Before defining runtime pipelines, graph processing or expression resolution, contributors must first understand what the Composition Engine represents within Mosaic.

Most UI frameworks answer the question:

"How should this interface be rendered?"

Mosaic intentionally answers a different question.

"What should the user understand right now?"

The Composition Engine exists to answer that question continuously.

It is not a layout engine.

It is not a widget framework.

It is not a rendering library.

It is an **understanding engine**.

Philosophy Statement

The interface is never authored. It is continuously solved from the user's current World.

Everything within the Composition Engine derives from this statement.

Why The Engine Exists

Traditional software typically follows this model.

[text]
Data

↓

Templates

↓

Components

↓

Render

Every user receives essentially the same interface.

Mosaic intentionally rejects this assumption.

Instead.

[text]
World

↓

Behaviour

↓

Composition

↓

Expressions

↓

Presentation

↓

Render

The interface becomes the consequence of understanding.

Not the starting point.

The Engine Solves Worlds

The Composition Engine does not solve layouts.

It solves Worlds.

A World already contains:

- information,
- relationships,
- hierarchy,
- behaviour,
- context.

The engine simply determines the most understandable way to communicate that World.

Behaviour Before Rendering

Rendering is the final stage.

Never the first.

Incorrect.

[text]
Screen

↓

Populate Widgets

↓

Display

Preferred.

[text]
Behaviour

↓

Composition

↓

Expressions

↓

Presentation

↓

Rendering

Rendering technology should remain almost irrelevant to the conceptual architecture.

Runtime Is Continuous

The Composition Engine never stops.

Traditional interfaces frequently behave like this.

[text]
Open Screen

↓

Render

↓

Finished

Mosaic instead behaves like this.

[text]
World Changes

↓

Composition Evolves

↓

Presentation Evolves

↓

World Changes Again

The runtime continuously accompanies the user.

The World Is Primary

Everything begins with the user's World.

Examples.

Watching

↓

Episode

Reading

↓

Book

Listening

↓

Album

The engine never asks:

"Which screen should I open?"

Instead it asks:

"What World currently exists?"

Components Are Incidental

Components are implementation.

The engine reasons about:

- Information,
- Expressions,
- Relationships,
- Behaviour.

Only after those concepts have been solved are components selected.

This separation dramatically increases long-term flexibility.

Expressions Are The Bridge

The Composition Engine never produces widgets.

It produces Expressions.

Examples.

Timeline

Progress

Hero

Metadata

Relationships

The Tile Framework later determines how those Expressions become visible.

Expressions therefore become the shared language between runtime behaviour and presentation.

Behaviour Creates Composition

Every runtime update begins with behaviour.

Examples.

Playback starts.

↓

Composition changes.

Episode ends.

↓

Composition changes.

Search opens.

↓

Composition changes.

Behaviour remains the highest authority throughout the runtime architecture.

Understanding Before Performance

Performance matters.

Understanding matters more.

The Composition Engine should always preserve:

- hierarchy,
- continuity,
- behavioural correctness,

before pursuing optimisation.

Optimisation should never alter conceptual correctness.

Adaptive By Default

The engine assumes that:

- devices change,
- layouts change,
- accessibility changes,
- runtime context changes.

Adaptation is therefore considered the normal operating mode.

Static interfaces are treated as a special case.

Runtime Is Deterministic

Given identical:

- World,
- Behaviour,
- Context,
- Information,

the Composition Engine should produce identical Expressions.

Deterministic behaviour enables:

- predictable interaction,
- reproducible testing,
- caching,
- platform consistency.

One Runtime

Every Mosaic client shares one conceptual runtime.

Examples.

Desktop.

↓

Runtime.

Phone.

↓

Same Runtime.

Television.

↓

Same Runtime.

Voice.

↓

Same Runtime.

Only Presentation differs.

The World remains identical.

Plugins Enrich Worlds

Extensions do not build interfaces.

They enrich the World.

Plugins contribute:

- information,
- relationships,
- artwork,
- behaviour.

The Composition Engine determines:

- hierarchy,
- expressions,
- runtime presentation.

This separation ensures every extension naturally feels native.

Runtime Is Invisible

Users should never perceive:

- runtime solving,
- composition graphs,
- expression resolution.

They should simply feel that:

The interface always understands what they need next.

The strongest runtime engine is the one users never realise exists.

Good Examples

Watching

Playback progresses.

↓

Timeline updates.

↓

Continue Watching evolves.

↓

Environment responds.

The World naturally continues.

Reading

Current chapter advances.

↓

Reading progress updates.

↓

Bookmarks adapt.

↓

Nothing feels rebuilt.

The experience quietly evolves.

Music

Track changes.

↓

Album becomes Hero.

↓

Playback continues.

↓

Atmosphere follows.

The transition feels inevitable.

Anti-patterns

Screen Thinking

Beginning implementation from pages.

Component Thinking

Designing widgets before understanding.

Layout Thinking

Treating layout as the primary runtime concern.

Plugin Interfaces

Allowing extensions to bypass runtime composition.

Composition Engine Model

```
[mermaid]
flowchart TD
```

```
World
World --> Behaviour
Behaviour --> CompositionSolver
CompositionSolver --> Expressions
Expressions --> PresentationModel
PresentationModel --> Rendering
Rendering --> Understanding
```

The engine exists to transform Worlds into understanding.

Rendering is simply the final implementation step.

Relationship To Future Chapters

The remaining chapters define how this philosophy becomes runtime architecture.

Including:

- Runtime World
- Composition Solver
- Expression Resolution
- Runtime Hierarchy
- Adaptive Layout
- Behaviour Orchestration
- Runtime Pipelines
- Composition Caching

Every implementation decision should reinforce the philosophy established here.

Summary

The Composition Engine is the architectural heart of Mosaic.

It continuously transforms:

- behaviour,
- information,
- relationships,
- context,

into one coherent user experience.

Users should never feel that screens are appearing.

They should feel that their World is naturally evolving around them.

That uninterrupted evolution is the defining objective of the Composition Engine.

Review Status

Status

Draft

Next File

02-runtime-world.md

Runtime World

Purpose

Everything within Mosaic begins with one concept.

The **World**.

Previous MDL specifications defined the World conceptually.

This chapter defines how that World exists at runtime.

The Runtime World is the authoritative representation of the user's current experience.

It is **not** a database.

It is **not** application state.

It is **not** a view model.

It is the living model from which every Composition is continuously solved.

Definition

Within MDS, the **Runtime World** is defined as:

The continuously evolving runtime representation of the user's current reality from which all behaviour, composition and presentation are derived.

Everything the user experiences originates here.

Why A Runtime World Exists

Traditional applications frequently construct interfaces from application state.

Examples.

[text]
Database

↓

API

↓

State

↓

UI

Mosaic intentionally follows a different model.

[text]
World

↓

Behaviour

↓

Composition

↓

Presentation

The Runtime World represents meaning.

Application state represents implementation.

These are intentionally different concepts.

One World

A Mosaic session possesses one Runtime World.

Not:

- Home World
- Playback World
- Search World
- Settings World

Instead.

World

↓

Current Behaviour

↓

Current Context

↓

Current Focus

The World evolves.

It is never replaced.

Runtime World Contents

The Runtime World contains conceptual information.

Examples include:

[text]
Focus

Context

Relationships

Behaviour

Identity

Timeline

Capabilities

Environment

Notice what is absent.

- widgets
- layouts
- screens
- components

The Runtime World remains presentation independent.

World Is Behavioural

The Runtime World is defined by behaviour.

Example.

Watching

↓

Episode

↓

Playback

↓

Paused

The World changes because behaviour changed.

Not because the interface changed.

World Is Continuous

The Runtime World should never reset unnecessarily.

Example.

Playback

↓

Search

↓

Playback

Search temporarily changes Context.

The World remains continuous.

Users should always feel they returned to the same experience.

World Is Shared

Every subsystem consumes the same Runtime World.

Examples.

Composition Engine.

↓

World.

Motion.

↓

World.

Colour.

↓

World.

Materials.

↓

World.

Typography.

↓

World.

The Runtime World becomes the single source of behavioural truth.

World Owns Focus

Only the Runtime World determines current Focus.

Examples.

Current Film

Current Book

Current Album

Other systems consume Focus.

They never redefine it.

World Owns Context

The Runtime World also owns Context.

Examples.

Browsing

Watching

Reading

Searching

Managing

Context determines:

- Composition
- Motion
- Density
- Atmosphere

Presentation follows afterwards.

World Owns Relationships

Relationships are considered first-class runtime concepts.

Examples.

Episode

↓

Series

↓

Franchise

↓

Studio

The Composition Engine consumes these relationships while solving understanding.

Relationships should never be reconstructed by components.

World Is Observable

Every behavioural system observes the Runtime World.

Conceptually.

[text]

Runtime World

↓

Behaviour

↓

Composition

↓

Presentation

Changes propagate naturally.

No subsystem should poll independently.

World Mutations

The Runtime World evolves through behavioural mutations.

Examples.

Playback Started

Playback Paused

Focus Changed

Episode Completed

Mutations should describe behavioural events.

Never implementation details.

Deterministic Evolution

Given identical behavioural events...

The Runtime World should evolve identically.

Example.

Watch Episode

↓

Pause

↓

Resume

↓

Complete

Every Mosaic client should produce the same Runtime World.

Presentation may differ.

Behaviour must not.

Runtime Ownership

Only the Runtime Engine should mutate the Runtime World.

Applications.

↓

Observe.

Plugins.



Contribute Behaviour.

Composition.



Consume.

Presentation.



Render.

Ownership should remain centralised.

World Snapshots

Future implementations may expose immutable Runtime World snapshots.

Conceptually.

[text]
Runtime World



Snapshot



Composition Solver



Presentation

Snapshots improve:

- determinism
- testing
- caching
- replayability

The live World continues evolving independently.

World Lifetime

The Runtime World exists for the lifetime of the user session.

Individual Compositions come and go.

Materials evolve.

Atmosphere changes.

The World persists.

Users should therefore experience one continuous reality rather than a series of disconnected screens.

Multi-User Worlds

Future Mosaic capabilities may support:

- shared households
- collaborative sessions
- watch parties

Each participant should possess an independent Runtime World.

Shared behaviour should synchronise through explicit runtime events.

The World model intentionally supports this evolution.

Plugins

Extensions contribute to the Runtime World.

Examples include:

- metadata
- artwork
- relationships
- behavioural events

Plugins never directly modify:

- Composition
- Hierarchy
- Presentation

The Runtime World remains the only behavioural authority.

Good Examples

Playback

Behaviour.

↓

Playback progresses.

↓

Runtime World updates.



Composition evolves.



Presentation updates.

Everything remains continuous.

Reading

Chapter changes.



Runtime World updates.



Progress evolves.



Reader continues.

No interface reconstruction occurs.

Music

Track changes.



Album remains Focus.



Playback Context evolves.



Atmosphere adapts.

The World remains coherent.

Anti-patterns

UI State World

Treating widget state as the Runtime World.

Multiple Worlds

Creating independent runtime models for different screens.

Component Ownership

Components mutating behavioural state.

Runtime Reset

Reconstructing the World whenever navigation changes.

Runtime World Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> RuntimeWorld
RuntimeWorld --> CompositionSolver
CompositionSolver --> Expressions
Expressions --> Presentation
Presentation --> Rendering
```

Every runtime system begins with the World.

Everything else is derived.

Relationship To Future Chapters

The next chapter defines the **Composition Solver**.

The Runtime World explains:

What exists.

The Composition Solver explains:

How that World becomes understandable.

Together they establish the runtime intelligence that distinguishes Mosaic from conventional interface frameworks.

Summary

The Runtime World is the living behavioural model of the user's experience.

It is the single source of truth for:

- Focus,
- Context,
- Relationships,
- Behaviour,
- Continuity.

Every runtime system observes it.

None redefine it.

The Runtime World is therefore the foundation upon which the entire Mosaic runtime architecture is built.

Review Status

Status

Draft

Next File

03-composition-solver.md

Composition Solver

Purpose

The Runtime World represents everything the user currently knows and experiences.

The Composition Solver transforms that World into understanding.

It is the decision-making core of the Composition Engine.

Unlike traditional layout engines, the Composition Solver does not determine:

- grids,
- rows,
- columns,
- widgets.

Instead it determines:

- importance,
- relationships,
- hierarchy,
- expressions,
- behavioural presentation.

Every runtime experience produced by Mosaic begins here.

Definition

Within MDS, the **Composition Solver** is defined as:

The deterministic runtime system responsible for transforming the Runtime World into the optimal Composition for the user's current behavioural context.

The Solver produces understanding.

Presentation is derived afterwards.

Why A Solver Exists

Traditional applications generally solve interfaces.

[text]
Data

↓

Templates

↓

Layout

↓

Render

Mosaic intentionally solves understanding.

[text]

Runtime World

↓

Composition Solver

↓

Expressions

↓

Presentation

The Solver therefore replaces manually authored interface structure with behavioural reasoning.

The Solver Never Designs

The Composition Solver should never make aesthetic decisions.

It does not determine:

- colour,
- typography,
- materials,
- spacing.

Those systems already exist.

The Solver determines:

What deserves attention?

Everything else follows naturally.

Inputs

The Solver consumes one Runtime World snapshot.

Conceptually.

[text]

Runtime World

↓

Focus

↓

Context

↓

Behaviour

↓

Relationships

↓

Information

↓

Capabilities

These become the complete behavioural inputs for solving.

No rendering information is required.

Outputs

The Solver produces a solved Composition.

Outputs include:

[text]
Hero

↓

Hierarchy

↓

Expressions

↓

Priority

↓

Grouping

↓

Behavioural Intent

↓

Presentation Model

Every downstream runtime system consumes these outputs.

Deterministic Solving

The Composition Solver must always remain deterministic.

Given identical:

- Runtime World,
- Behaviour,

- Context,
- Relationships,

the Solver should produce identical Composition.

This enables:

- predictable behaviour,
- runtime caching,
- replay,
- testing,
- cross-platform consistency.

Behaviour Is Authority

Behaviour always possesses highest authority.

Examples.

Playback begins.

↓

Playback becomes primary.

Hero changes.

↓

Composition reorganises.

Search opens.

↓

Overlay Composition appears.

Behaviour determines composition.

The Solver simply communicates it.

Focus Resolution

Only one Focus should normally exist.

Example.

Watching Frieren

The Solver identifies:

Hero

↓

Frieren

↓

Episode 18

Everything else becomes supporting information.

Focus should emerge naturally from the Runtime World.

Relationship Resolution

Relationships significantly influence Composition.

Example.

Episode

↓

Season

↓

Series

↓

Manga

↓

Author

The Solver evaluates:

- proximity,
- behavioural relevance,
- contextual usefulness.

Relationships strengthen understanding.

Not complexity.

Priority Resolution

Priority should emerge from behaviour.

Conceptually.

[text]

Behaviour

↓

Priority

↓

Hierarchy

↓

Expressions

Priority should never be inferred from:

- popularity,
- layout,
- implementation.

Understanding always possesses higher authority.

Expression Resolution

The Solver never produces components.

Instead it produces Expressions.

Example.

Continue Watching

↓

Timeline

↓

Relationships

↓

Metadata

↓

Actions

Later specifications determine how these Expressions become Tiles and Components.

The Solver remains presentation independent.

Group Resolution

Related information should naturally group together.

Example.

Current Episode

Progress

Next Episode

Rather than:

Current Episode

Codec

Progress

Reviews

Next Episode

Grouping reduces cognitive effort.

It should emerge from relationships rather than manual interface design.

Adaptive Solving

The Solver should continuously adapt.

Behaviour changes.

↓

Composition evolves.

↓

Expressions evolve.

↓

Presentation evolves.

The engine should never require predefined layouts for every possible situation.

Local Solving

Small behavioural changes should produce small compositional changes.

Example.

Playback progress updates.

↓

Timeline updates.

The Hero remains unchanged.

The Composition should avoid unnecessary recomputation whenever practical.

Runtime Graph

Future implementations may represent the Runtime World internally as a graph.

Conceptually.

[text]
Runtime Graph

↓

Solver

↓

Composition

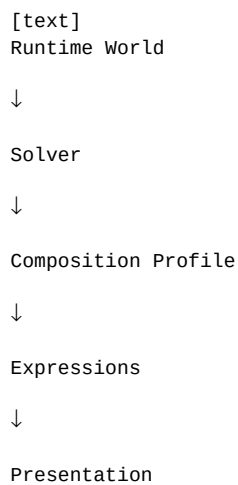
The graph remains an implementation detail.

The conceptual architecture defined here should remain unchanged regardless of runtime representation.

Composition Profiles

Future runtime implementations may internally construct Composition Profiles.

Conceptually.



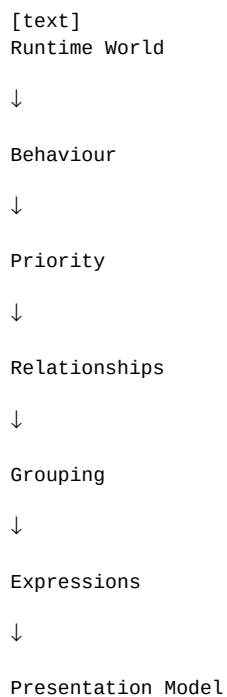
Profiles improve:

- caching,
- determinism,
- incremental updates.

Applications should remain unaware of their existence.

Solver Pipeline

Conceptually.



Every stage contributes exactly one responsibility.

Runtime Updates

Typical Solver triggers include:

- Focus changes
- Behaviour changes
- Context changes
- Relationship updates
- Plugin contributions
- User interaction

Minor visual changes should not invoke complete recomposition.

The Solver should favour incremental behavioural evolution.

Plugins

Extensions contribute:

- information,
- relationships,
- behaviours.

Plugins never contribute:

- hierarchy,
- expressions,
- layouts,
- presentation.

The Composition Solver remains the sole authority for organising understanding.

Good Examples

Playback

Behaviour.

↓

Episode playing.

↓

Timeline promoted.

↓

Progress updated.

↓

Presentation evolves.

The user immediately understands current activity.

Reading

Current Chapter.

↓

Bookmarks.

↓

Reading Progress.

↓

Supporting Information.

Everything naturally supports the reader.

Music

Album.

↓

Current Track.

↓

Playback Queue.

↓

Related Albums.

The Composition reflects listening behaviour rather than library structure.

Anti-patterns

Screen Solver

Producing pages instead of understanding.

Widget Solver

Selecting components before Expressions.

Layout Solver

Beginning with grids rather than hierarchy.

Plugin Composition

Allowing extensions to determine presentation.

Composition Solver Model

[mermaid]
flowchart TD

```
RuntimeWorld
RuntimeWorld --> Behaviour
Behaviour --> Priority
Priority --> Relationships
Relationships --> Grouping
Grouping --> Expressions
Expressions --> PresentationModel
```

The Composition Solver transforms behavioural reality into understandable experience.

Everything else becomes implementation.

Relationship To Future Chapters

The next chapter defines **Expression Resolution**.

The Composition Solver explains:

What should exist.

Expression Resolution explains:

How those solved concepts become reusable runtime expressions.

Together they form the conceptual core of the Composition Engine.

Summary

The Composition Solver is the runtime intelligence of Mosaic.

It continuously transforms:

- behaviour,
- information,
- relationships,
- context,

into one coherent understanding of the user's current World.

The interface is therefore never manually designed.

It is continuously solved.

That distinction is the defining architectural characteristic of the Mosaic Composition Engine.

Review Status

Status

Draft

Next File

04-expression-resolution.md

Expression Resolution

Purpose

The Composition Solver determines **what should be understood**.

Expression Resolution determines **how that understanding should be communicated**.

This distinction is fundamental.

The Solver never creates interface.

It produces conceptual Expressions.

Expression Resolution transforms those conceptual Expressions into a platform-independent Presentation Model.

Only after this stage do components become relevant.

Definition

Within MDS, **Expression Resolution** is defined as:

The deterministic process through which solved concepts become reusable runtime Expressions suitable for presentation.

Expression Resolution translates understanding into communication.

It does not translate understanding into components.

Why Expressions Exist

Traditional UI frameworks often follow this process.

[text]
Data

↓

Widgets

↓

Render

Mosaic intentionally introduces an intermediate layer.

[text]
Behaviour

↓

Composition

↓

Expressions

↓

Presentation

↓

Components

Expressions preserve conceptual meaning while remaining independent from implementation.

Expressions Are Behavioural

Expressions communicate behaviour.

Not appearance.

Examples include:

[text]
Hero

Timeline

Progress

Relationships

Metadata

Actions

Queue

Search

Playback

Notice that none of these describe:

- buttons
- cards
- shelves
- grids

Expressions communicate concepts.

One Expression

An Expression represents one conceptual responsibility.

Poor.

[text]
HeroCardWithButtons

Several responsibilities have merged.

Preferred.

[text]
Hero

↓

Actions

Each Expression communicates one idea.

The Presentation Model later determines how those ideas are visually combined.

Inputs

Expression Resolution consumes:

[text]
Composition

↓

Hierarchy

↓

Priority

↓

Grouping

↓

Behaviour

The Runtime World is no longer consulted directly.

The Composition has already solved understanding.

Expression Resolution simply communicates it.

Outputs

Expression Resolution produces:

[text]
Presentation Model

↓

Expression Tree

↓

Presentation Metadata

↓

Interaction Model

↓

Material Intent

These outputs remain completely independent from rendering technologies.

Expression Tree

Future implementations may internally represent Expressions as a tree.

Conceptually.

[text]
Hero

■■■ Progress

■■■ Actions

■■■ Metadata

■■■ Relationships

This tree represents conceptual communication.

Not interface structure.

A renderer may later choose:

- tiles,
- lists,
- overlays,
- voice output.

The Expression Tree remains identical.

Expression Identity

Expressions possess stable identities.

Example.

[text]
Timeline

The Timeline may appear as:

- horizontal shelf,
- vertical list,
- compact mobile view,
- television timeline,
- spoken voice summary.

It remains:

Timeline

Identity survives implementation.

Behaviour Preservation

Expressions inherit behavioural meaning.

Example.

Playback

↓

Progress

↓

Timeline

Every implementation should preserve this relationship.

The renderer should never reinterpret behavioural ordering.

Composition Awareness

Expression Resolution respects Composition.

Example.

Hero.

↓

Resolved first.

Supporting.

↓

Resolved second.

Peripheral.

↓

Resolved last.

Expression order reflects behavioural hierarchy.

Not rendering convenience.

Material Intent

Expression Resolution also produces Material Intent.

Examples.

[text]

Hero

↓

Hero Material

[text]

Overlay

↓

Overlay Material

[text]
Navigation

↓

Surface Material

The Material System later resolves these into physical behaviour.

Expression Resolution communicates intention.

Not rendering.

Typography Intent

Expressions also communicate editorial intent.

Examples.

[text]
Hero

↓

Heading

[text]
Metadata

↓

Supporting

[text]
Diagnostics

↓

Caption

Typography therefore becomes another consequence of Expression Resolution.

Motion Intent

Expressions also inherit Motion behaviour.

Examples.

Hero.

↓

Hero Motion.

Overlay.

↓

Overlay Motion.

Timeline.



Supporting Motion.

Motion therefore follows Expressions naturally.

Components never invent movement independently.

Adaptive Expressions

Expressions remain adaptive.

Example.

Desktop.

[text]
Timeline



Expanded Expression

Phone.

[text]
Timeline



Compact Expression

Voice.

[text]
Timeline



Spoken Expression

The conceptual identity remains unchanged.

Only presentation evolves.

Expression Composition

Expressions may contain other Expressions.

Example.

[text]
Hero

■■■ Artwork

■■■ Heading

■■■ Progress

■■■ Actions

■■■ Metadata

Each child remains independently meaningful.

Future implementations may reorganise them without changing behavioural understanding.

Runtime Updates

Expressions should update incrementally.

Example.

Progress changes.

↓

Progress Expression updates.

The Hero remains stable.

Incremental updates preserve continuity while reducing runtime work.

Deterministic Resolution

Given identical Composition.

↓

Expression Resolution should always produce identical Expression Trees.

Determinism enables:

- caching,
- replay,
- testing,
- cross-platform consistency.

Plugins

Extensions contribute:

- information,
- relationships,
- behaviours.

Plugins never contribute Expressions.

The Composition Engine determines which Expressions exist.

Every extension therefore inherits the same conceptual language.

Good Examples

Playback

Composition.



Hero.



Progress.



Timeline.



Presentation.

The user immediately understands current playback.

Reading

Book.



Chapter.



Bookmarks.



Reading Progress.

Editorial understanding naturally emerges.

Music

Album.



Current Track.



Playback Queue.



Recommendations.

Expressions communicate listening behaviour.

Anti-patterns

Component Resolution

Generating widgets directly from Composition.

Platform Expressions

Different platforms inventing different conceptual Expressions.

Plugin Expressions

Extensions defining custom interface structures.

Layout Thinking

Expression identity depending upon grid position.

Expression Resolution Model

[mermaid]
flowchart TD

```
Composition
Composition --> ExpressionResolution
ExpressionResolution --> ExpressionTree
ExpressionTree --> PresentationModel
PresentationModel --> Rendering
```

Expressions bridge understanding and presentation.

They intentionally remain independent from rendering.

Relationship To Future Chapters

The next chapter defines **Runtime Hierarchy**.

Expression Resolution explains:

What should be communicated.

Runtime Hierarchy explains:

How those Expressions are prioritised continuously as behaviour evolves.

Together they establish the runtime language through which the user's World becomes visible.

Summary

Expression Resolution transforms solved understanding into reusable runtime communication.

It separates:

- behaviour
- understanding
- presentation

allowing every Mosaic client to present one consistent World regardless of rendering technology.

Expressions are therefore one of the most important architectural abstractions within the entire Mosaic platform.

Review Status

Status

Draft

Next File

05-runtime-hierarchy.md

Runtime Hierarchy

Purpose

The Composition Solver determines what should exist.

Expression Resolution determines how those concepts become communicable.

Runtime Hierarchy determines **how those Expressions continuously relate to one another as the user's World evolves**.

Unlike traditional interfaces, hierarchy within Mosaic is never static.

It is continuously resolved from behaviour.

The user should therefore experience one interface that naturally adapts without ever feeling unpredictable.

Definition

Within MDS, **Runtime Hierarchy** is defined as:

The continuously evaluated ordering of Expressions according to the user's current World, Behaviour and Composition.

Runtime Hierarchy is behavioural.

Not visual.

Presentation communicates hierarchy.

It does not create it.

Why Runtime Hierarchy Exists

Traditional applications frequently define hierarchy statically.

Example.

[text]
Header

↓

Sidebar

↓

Content

↓

Footer

This hierarchy exists regardless of:

- user intent
- behaviour

- context

Mosaic intentionally rejects this model.

Instead.

[text]
Behaviour

↓

Priority

↓

Hierarchy

↓

Expressions

Hierarchy becomes the consequence of behaviour.

Hierarchy Is Continuous

Runtime Hierarchy should evolve continuously.

Example.

Browsing

↓

Hero

↓

Watching

↓

Playback

↓

Episode Complete

↓

Continue Watching

Nothing has been rebuilt.

Hierarchy simply evolved.

Users should perceive continuity rather than transition.

Runtime Inputs

Runtime Hierarchy evaluates:

[text]
Behaviour

↓

Focus

↓

Context

↓

Priority

↓

Relationships

↓

Composition

Rendering information is intentionally absent.

Hierarchy exists before presentation.

Runtime Outputs

Runtime Hierarchy produces:

[text]

Hero

↓

Primary

↓

Supporting

↓

Peripheral

↓

Background

Every Expression receives exactly one current hierarchical role.

Future runtime systems consume this ordering.

Hero Resolution

Exactly one Hero should normally exist.

Example.

Playback

↓

Episode

The Hero receives:

- strongest editorial emphasis
- highest material quality
- primary motion
- greatest atmospheric influence

Everything else naturally organises itself around the Hero.

Dynamic Priority

Priority changes.

Hierarchy follows.

Example.

Current activity.

Browsing Reviews

Reviews become:

Primary

Playback information becomes:

Supporting

Nothing about the information changed.

Only its behavioural importance.

Relationship Influence

Relationships influence Runtime Hierarchy.

Example.

Episode

↓

Next Episode

↓

Series

↓

Franchise

The closer a relationship is to the user's current behaviour...

The stronger its hierarchical influence should become.

Hierarchy should therefore emerge naturally from the World rather than fixed navigation structures.

Local Hierarchy

Hierarchy should evolve locally whenever practical.

Example.

Playback progress updates.

↓

Timeline hierarchy updates.

Hero remains unchanged.

The remainder of the Composition should remain behaviourally stable.

This preserves continuity while minimising unnecessary runtime work.

Environmental Hierarchy

Materials also participate in Runtime Hierarchy.

Examples.

Hero Material.

↓

Highest physical presence.

Supporting Acrylic.

↓

Moderate presence.

Canvas.

↓

Stable environment.

Material Hierarchy should always reinforce Runtime Hierarchy.

Never compete with it.

Editorial Hierarchy

Typography inherits Runtime Hierarchy.

Examples.

Hero.

↓

Heading.

Supporting.

↓

Body.

Peripheral.

↓

Caption.

Typography therefore communicates behavioural importance without requiring independent editorial decisions.

Motion Hierarchy

Motion also follows Runtime Hierarchy.

Example.

Hero changes.

↓

Hero moves first.

↓

Supporting Expressions respond.

↓

Environment settles.

Movement therefore becomes another consequence of hierarchy.

Runtime Stability

Hierarchy should change only when behaviour justifies it.

Poor.

Minor Layout Change

↓

Hierarchy Changes

Preferred.

Behaviour Changes

↓

Hierarchy Evolves

Visual adaptation alone should never reorganise conceptual importance.

Multi-Device Hierarchy

Every device should share identical Runtime Hierarchy.

Desktop.



Expanded Presentation.

Phone.



Compact Presentation.

Television.



Immersive Presentation.

The hierarchy remains identical.

Only expression changes.

Accessibility

Accessibility should never change Runtime Hierarchy.

Examples.

Large text.



Hierarchy preserved.

Reduced motion.



Hierarchy preserved.

High contrast.



Hierarchy preserved.

Accessibility modifies presentation.

Never behavioural importance.

Runtime Profiles

Future implementations may internally represent Runtime Hierarchy through Hierarchy Profiles.

Conceptually.

[text]

Composition

↓

Hierarchy Profile

↓

Expression Ordering

↓

Presentation

Profiles improve:

- caching
- deterministic updates
- incremental recomposition

Applications should remain unaware of their existence.

Runtime Updates

Typical hierarchy recalculation triggers include:

- Focus changes
- Behaviour changes
- Context changes
- Relationship updates
- Plugin contributions

Ordinary rendering updates should not trigger hierarchy recalculation.

Plugins

Extensions contribute:

- behaviours
- information
- relationships

Plugins never assign:

- Hero
- Priority
- Hierarchy

The Runtime Hierarchy remains entirely platform owned.

Every extension therefore naturally inherits one behavioural language.

Good Examples

Playback

Behaviour.



Playback.



Hero.



Timeline.



Supporting metadata.

Everything feels inevitable.

Reading

Book.



Current Chapter.



Reading Progress.



Bookmarks.

Editorial hierarchy emerges naturally.

Music

Album.



Current Track.



Queue.



Recommendations.

Listening behaviour determines importance.

Anti-patterns

Static Hierarchy

Fixed interface hierarchy regardless of behaviour.

Layout Hierarchy

Screen position determining importance.

Component Hierarchy

Widgets deciding priority.

Plugin Hierarchy

Extensions assigning their own behavioural importance.

Runtime Hierarchy Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> Priority
Priority --> RuntimeHierarchy
RuntimeHierarchy --> Expressions
Expressions --> PresentationModel
PresentationModel --> Rendering
```

Behaviour determines hierarchy.

Hierarchy determines communication.

Rendering simply expresses the result.

Relationship To Future Chapters

The next chapter defines **Adaptive Layout**.

Runtime Hierarchy explains:

What deserves attention.

Adaptive Layout explains:

How that hierarchy is expressed across different devices and environments without changing behavioural meaning.

Together they complete the behavioural foundation of runtime presentation.

Summary

Runtime Hierarchy continuously transforms behavioural importance into understandable structure.

It ensures that:

- the Hero remains obvious,
- supporting information remains supportive,
- peripheral information stays quiet,
- every Mosaic client communicates the same behavioural language.

Hierarchy is therefore never authored.

It is continuously solved from the evolving World.

Review Status

Status

Draft

Next File

06-adaptive-layout.md

Adaptive Layout

Purpose

The Composition Solver determines:

- what should exist,
- what deserves attention,
- how understanding should be organised.

Adaptive Layout determines how that solved understanding is expressed within the physical constraints of a device.

Unlike responsive design, Adaptive Layout does **not** solve geometry first.

It solves understanding first.

Layout becomes a consequence.

Definition

Within MDS, **Adaptive Layout** is defined as:

The deterministic projection of a solved Composition into a device-specific spatial arrangement while preserving behavioural hierarchy and understanding.

Adaptive Layout changes:

- spatial organisation,
- density,
- presentation.

It never changes:

- hierarchy,
- behaviour,
- meaning.

Why Adaptive Layout Exists

Traditional responsive systems generally behave like this.

[text]
Viewport

↓

Breakpoints

↓

Different Layout

Meaning frequently changes because layout changed.

Mosaic intentionally follows:

[text]
Composition

↓

Hierarchy

↓

Expressions

↓

Adaptive Layout

↓

Presentation

Understanding remains constant.

Only spatial expression changes.

Behaviour Before Geometry

Adaptive Layout should never ask:

How much space exists?

before asking:

What must remain understandable?

Behaviour always possesses higher authority than geometry.

One Composition

Every device should consume the same solved Composition.

Desktop.

↓

Expanded Presentation.

Tablet.

↓

Editorial Presentation.

Phone.

↓

Compact Presentation.

Television.

↓

Immersive Presentation.

The Composition never changes.

Only layout.

Layout Inputs

Adaptive Layout evaluates:

[text]
Expression Tree

↓

Hierarchy

↓

Device Class

↓

Orientation

↓

Viewing Distance

↓

Capabilities

Notice that Behaviour has already been solved.

Adaptive Layout communicates.

It does not decide.

Layout Outputs

Adaptive Layout produces:

[text]
Spatial Regions

↓

Expression Placement

↓

Density

↓

Presentation Constraints

These outputs remain independent from components.

Future rendering systems consume them directly.

Regions

Adaptive Layout organises Expressions into conceptual regions.

Examples.

[text]
Hero

↓

Primary Region

↓

Supporting Region

↓

Peripheral Region

↓

Navigation

Regions communicate behavioural organisation.

Not implementation.

Density

Adaptive Layout may alter density.

Examples.

Desktop.

↓

Expanded spacing.

Phone.

↓

Compact spacing.

Television.

↓

Large spacing.

The hierarchy remains identical.

Only physical density changes.

Progressive Disclosure

Smaller devices should prefer progressive disclosure over hierarchy reduction.

Incorrect.

[text]
Remove Relationships

Preferred.

[text]
Relationships

↓

Collapsed

↓

Expandable

Understanding remains available.

Presentation simply becomes more compact.

Hero Preservation

The Hero should remain visually dominant across every layout.

Examples.

Desktop.

↓

Large Hero region.

Phone.

↓

Compact Hero.

Television.

↓

Immersive Hero.

The Hero should always remain immediately recognisable.

Anchor Preservation

Anchors should remain behaviourally stable.

Examples include:

- Navigation
- Search
- Playback
- Current Focus

Adaptive Layout may reposition Anchors.

It should never redefine them.

Expression Integrity

Expressions should never fragment because layout changes.

Example.

Timeline.



Timeline.

Not.

Timeline Header.



Timeline Progress.



Timeline Footer.

Expressions remain conceptually whole even when visually rearranged.

Material Awareness

Adaptive Layout should preserve Material Hierarchy.

Hero Region.



Hero Material.

Supporting Region.



Acrylic.

Canvas.



Environment.

Layout should never weaken physical hierarchy.

Typography Awareness

Editorial hierarchy should remain stable.

Heading.



Heading.

Body.

↓

Body.

Supporting.

↓

Supporting.

Adaptive Layout may alter line length and spacing.

It should never alter editorial roles.

Motion Awareness

Adaptive Layout should preserve behavioural sequencing.

Examples.

Hero moves first.

↓

Supporting Expressions respond.

↓

Environment settles.

Changing layout should never create a different Motion language.

Device Classes

Future runtime implementations may define conceptual device classes.

Examples.

[text]
Phone

Tablet

Desktop

Television

Voice

Each device class receives its own adaptive layout strategy.

All consume identical Expressions.

Runtime Adaptation

Adaptive Layout should respond to:

- orientation changes,
- window resizing,
- foldable devices,
- accessibility scaling.

These changes should preserve continuity.

Users should feel the interface adapting.

Not rebuilding.

Incremental Layout

Small environmental changes should produce small layout changes.

Preferred.

[text]
Window Slightly Wider

↓

Additional Supporting Expression Appears

Avoid.

[text]
Window Slightly Wider

↓

Entire Layout Rebuilt

Incremental adaptation preserves orientation.

Accessibility

Accessibility should influence layout.

Examples.

Larger text.

↓

Greater spacing.

Reduced vision.

↓

Simpler layout.

High contrast.

↓

Unchanged hierarchy.

Accessibility modifies spatial expression.

Not behavioural meaning.

Plugins

Extensions contribute Expressions.

Plugins never determine layout.

Adaptive Layout remains entirely platform owned.

Every extension therefore automatically inherits future layout improvements.

Good Examples

Desktop

Hero.



Expanded supporting regions.



Peripheral collections.

Everything breathes.

Phone

Hero.



Primary actions.



Progressive disclosure.

Understanding remains intact.

Television

Large Hero.



Generous spacing.



Minimal interface.

Entertainment remains dominant.

Anti-patterns

Breakpoint Thinking

Entire interface changes because width crossed an arbitrary value.

Hierarchy Loss

Smaller devices removing behavioural importance.

Component Layout

Widgets determining spatial organisation.

Plugin Layout

Extensions introducing independent layout systems.

Adaptive Layout Model

[mermaid]
flowchart TD

```
Composition
Composition --> ExpressionTree
ExpressionTree --> AdaptiveLayout
AdaptiveLayout --> PresentationModel
PresentationModel --> Rendering
```

Composition determines understanding.

Adaptive Layout determines spatial expression.

Relationship To Future Chapters

The next chapter defines **Behaviour Orchestration**.

Adaptive Layout explains:

Where Expressions should appear.

Behaviour Orchestration explains:

How every runtime subsystem evolves together as behaviour changes.

Together they transform solved understanding into one continuously evolving runtime experience.

Summary

Adaptive Layout is not responsive design.

It is behavioural projection.

The user's World remains identical across:

- phones,
- desktops,
- televisions,
- future devices.

Only the physical arrangement changes.

That distinction allows Mosaic to remain one coherent Companion regardless of where users choose to experience it.

Review Status

Status

Draft

Next File

07-behaviour-orchestration.md

Behaviour Orchestration

Purpose

The Runtime World evolves continuously.

Every behavioural change affects multiple systems simultaneously.

Examples include:

- Composition
- Motion
- Materials
- Typography
- Atmosphere

If each subsystem reacted independently, the platform would feel fragmented.

Behaviour Orchestration ensures every subsystem evolves as one coherent behavioural experience.

The user should never perceive separate systems updating.

They should perceive one World continuing naturally.

Definition

Within MDS, **Behaviour Orchestration** is defined as:

The coordinated execution of every runtime subsystem in response to behavioural change while preserving continuity, determinism and understanding.

Orchestration is responsible for sequencing.

Not solving.

The Composition Solver determines *what* should happen.

Behaviour Orchestration determines *how every subsystem evolves together*.

Why Orchestration Exists

Traditional interfaces frequently behave like this.

[text]

State Changes

↓

Layout Updates

↓

Animation

↓

Theme Updates

↓

Done

Each subsystem behaves independently.

Mosaic intentionally behaves differently.

[text]

Behaviour

↓

Composition

↓

Expressions

↓

Materials

↓

Motion

↓

Presentation

↓

Understanding

The runtime behaves as one coordinated organism.

Behaviour Is The Trigger

Every orchestration cycle begins with behaviour.

Examples include:

[text]

Playback Started

Playback Paused

Focus Changed

Episode Completed

Search Opened

Chapter Changed

Rendering events should never initiate orchestration.

Behaviour always possesses the highest authority.

Orchestration Pipeline

Every behavioural event should follow the same conceptual pipeline.

[text]
Behaviour

↓

Runtime World

↓

Composition Solver

↓

Expression Resolution

↓

Hierarchy Update

↓

Presentation Model

↓

Motion

↓

Rendering

Each stage contributes one responsibility.

No stage should bypass another.

World Snapshot

Every orchestration cycle begins from one immutable Runtime World snapshot.

[text]
Runtime World

↓

Snapshot

↓

Entire Pipeline

This guarantees that every subsystem responds to identical behavioural information.

No subsystem should observe partially updated state.

Sequential Consistency

Subsystems should evolve in a predictable order.

Preferred order.

[text]
Behaviour

↓

Composition

↓

Expressions

↓

Materials

↓

Typography

↓

Motion

↓

Presentation

This ordering mirrors the architectural dependency chain established throughout MDL and MDS.

Atomic Behaviour

One behavioural event should produce one coherent runtime update.

Poor.

[text]
Playback

↓

Composition

↓

Later...

↓

Materials

↓

Later...

↓

Motion

Preferred.

[text]
Playback

↓

Entire Runtime Evolves Together

Users should never perceive intermediate states.

Incremental Evolution

Behaviour Orchestration should minimise unnecessary work.

Example.

Progress updates.

↓

Timeline Expression updates.

↓

Timeline Material updates.

↓

Timeline Motion updates.

The Hero remains unchanged.

Local behavioural changes should remain local.

Dependency Graph

Future implementations may internally construct a dependency graph.

Conceptually.

[text]
Behaviour

↓

Dependency Graph

↓

Affected Systems

↓

Presentation

The graph remains an implementation detail.

Behavioural ordering remains architecturally defined by this specification.

Behaviour Categories

Different behaviours naturally produce different orchestration paths.

Examples.

Focus Change

Updates:

- Composition
- Hero
- Expressions
- Materials
- Atmosphere
- Motion

Playback Progress

Updates:

- Timeline
- Progress
- Motion

Only.

Search

Updates:

- Overlay
- Navigation
- Interaction

The Runtime World itself remains largely unchanged.

Behaviour determines orchestration scope.

Material Coordination

Material systems should never update independently.

Preferred.

[text]
Composition

↓

Hero Material

↓

Atmosphere

↓

Refraction

↓

Presentation

Every Material subsystem should receive identical behavioural inputs.

Typography Coordination

Typography should participate after Composition has stabilised.

Reasons.

Editorial hierarchy depends upon:

- Hero
- Expressions
- Runtime Hierarchy

Typography should therefore respond.

Never lead.

Motion Coordination

Motion begins only after the behavioural model has been solved.

Motion communicates change.

It never determines change.

The Motion System therefore consumes the resolved Presentation Model.

Rendering Boundary

Rendering begins only after orchestration completes.

Conceptually.

[text]
Behaviour

↓

Runtime Systems

↓

Presentation Model

↓

Renderer

Rendering frameworks should remain passive consumers.

They should never influence orchestration.

Failure Behaviour

If one subsystem cannot update.

Preferred.

[text]
Behaviour

↓

Graceful Degradation

↓

Presentation Continues

Avoid.

[text]
Behaviour

↓

Entire Runtime Stops

The Composition Engine should remain resilient.

Individual subsystem failures should not prevent behavioural continuity wherever practical.

Deterministic Orchestration

Given identical:

- Runtime World
- Behaviour
- Context

the orchestration pipeline should always produce identical outputs.

Deterministic behaviour enables:

- replay
- debugging
- testing
- caching
- synchronisation

Every client should therefore experience equivalent runtime behaviour.

Multi-Device Behaviour

Different devices may render differently.

They should orchestrate identically.

Desktop.



Same Behaviour Pipeline.

Phone.



Same Behaviour Pipeline.

Television.



Same Behaviour Pipeline.

Presentation differs.

Behaviour does not.

Plugins

Extensions contribute:

- behaviours
- information
- relationships

Plugins never orchestrate runtime systems.

The Composition Engine determines:

- sequencing
- hierarchy
- subsystem execution

Every extension therefore inherits one coherent runtime architecture.

Good Examples

Playback

Playback begins.



Composition updates.



Timeline resolves.



Hero remains stable.

↓

Motion communicates change.

↓

Presentation updates.

Everything feels continuous.

Reading

Chapter changes.

↓

Progress updates.

↓

Bookmarks evolve.

↓

Typography remains stable.

↓

Reader continues naturally.

Music

Track changes.

↓

Hero updates.

↓

Playback queue adapts.

↓

Atmosphere redistributes.

↓

Environment settles.

The World quietly evolves.

Anti-patterns

Independent Systems

Every subsystem updates independently.

Renderer Ownership

Rendering engine driving behavioural updates.

Partial State

Subsystems observing different Runtime Worlds.

Behaviour Duplication

Multiple systems attempting to solve the same behavioural problem.

Behaviour Orchestration Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> RuntimeWorld
RuntimeWorld --> CompositionSolver
CompositionSolver --> ExpressionResolution
ExpressionResolution --> RuntimeHierarchy
RuntimeHierarchy --> PresentationModel
PresentationModel --> Motion
Motion --> Renderer
Renderer --> Understanding
```

One behavioural event.

One coordinated runtime evolution.

Relationship To Future Chapters

The next chapter defines **Runtime Pipelines**.

Behaviour Orchestration explains:

How runtime systems evolve together.

Runtime Pipelines explain:

How those coordinated stages are executed efficiently inside the Composition Engine.

Together they establish the execution architecture of the Mosaic runtime.

Summary

Behaviour Orchestration is the conductor of the Mosaic runtime.

Every subsystem:

- Composition,
- Materials,

- Typography,
- Motion,
- Presentation,

should evolve together from one shared behavioural understanding.

Users should never perceive multiple systems updating.

They should simply feel that their World naturally continued.

Review Status

Status

Draft

Next File

08-runtime-pipelines.md

Runtime Pipelines

Purpose

Behaviour Orchestration defines how runtime systems evolve together.

Runtime Pipelines define **how those behavioural stages execute efficiently inside the Composition Engine**.

The Composition Engine should not be viewed as one large process.

Instead it is a series of deterministic runtime pipelines.

Each pipeline performs one responsibility.

Together they continuously construct the user's World.

Definition

Within MDS, **Runtime Pipelines** are defined as:

The ordered execution stages through which behavioural changes are transformed into a fully resolved Presentation Model.

Pipelines execute architecture.

They do not define architecture.

Philosophy

Many runtime frameworks follow this model.

[text]
State

↓

Render

↓

Done

Mosaic intentionally follows:

[text]
Behaviour

↓

Pipeline

↓

Understanding

↓

Presentation

↓

Render

The runtime solves understanding before rendering begins.

Pipeline Principles

Every runtime pipeline should satisfy the following principles.

- deterministic
- incremental
- behaviour-driven
- presentation independent
- cacheable
- observable

Pipelines should communicate intent.

Not implementation.

Pipeline Overview

Every behavioural event should travel through the same conceptual pipeline.

[text]

Behaviour

↓

Runtime World

↓

Composition Solver

↓

Expression Resolution

↓

Hierarchy Resolution

↓

Adaptive Layout

↓

Presentation Model

↓

Rendering

No stage should be skipped.

No stage should duplicate another.

Stage One

Behaviour Intake

Purpose.

Receive behavioural events.

Examples.

- Playback Started
- Focus Changed
- Search Opened
- Chapter Completed

This stage validates behavioural intent.

It performs no presentation work.

Stage Two

Runtime World Update

Purpose.

Apply behavioural mutations to the Runtime World.

Outputs include:

- updated Focus
- updated Context
- updated Relationships

The Runtime World becomes the single source of truth for the remainder of the pipeline.

Stage Three

Composition Solver

Purpose.

Determine:

- Hero
- Priority
- Grouping
- Expressions

The Solver constructs understanding.

It produces no presentation.

Stage Four

Expression Resolution

Purpose.

Transform solved concepts into reusable Expressions.

Outputs include:

- Expression Tree
- Material Intent
- Typography Intent
- Motion Intent

Expressions remain implementation independent.

Stage Five

Runtime Hierarchy

Purpose.

Resolve behavioural importance.

Outputs include:

[text]
Hero

↓

Primary

↓

Supporting

↓

Peripheral

Every Expression receives exactly one current runtime role.

Stage Six

Adaptive Layout

Purpose.

Project Expressions into spatial organisation.

Adaptive Layout considers:

- device class
- orientation
- viewing distance
- accessibility

Behaviour remains unchanged.

Only spatial expression adapts.

Stage Seven

Presentation Model

Purpose.

Construct a platform-independent presentation description.

The Presentation Model contains:

- Expressions
- Regions
- Materials
- Typography
- Motion
- Interaction metadata

Components do not yet exist.

Stage Eight

Rendering

Rendering frameworks consume the Presentation Model.

Examples.

- Flutter
- Web
- SwiftUI
- Compose

Rendering should remain a passive implementation stage.

Behaviour has already been solved.

Incremental Pipelines

Not every behavioural event should execute every stage.

Example.

Playback progress.

Preferred.

[text]
Behaviour

↓

Timeline Expression

↓

Presentation Update

Avoid.

[text]
Behaviour

↓

Entire Pipeline

↓

Complete Rebuild

Incremental execution improves performance while preserving behavioural correctness.

Parallel Execution

Future implementations may execute independent stages concurrently.

Examples.

Typography.

↓

Parallel.

Materials.

↓

Parallel.

Motion.

↓

Parallel.

Provided they consume the same immutable Presentation Model.

Parallel execution should never compromise determinism.

Pipeline Snapshots

Every stage should consume immutable inputs.

Conceptually.

[text]
Snapshot

↓

Stage

↓

Snapshot

↓

Next Stage

Immutable snapshots improve:

- replay
- testing
- debugging
- caching

No stage should mutate data owned by another stage.

Failure Recovery

Pipelines should degrade gracefully.

Preferred.

[text]
Material Failure

↓

Fallback Material

↓

Continue

Avoid.

[text]
Material Failure

↓

Entire Runtime Stops

The user's World should remain available whenever practical.

Pipeline Caching

Pipeline stages should cache deterministic outputs.

Examples.

[text]
Runtime World

↓

Composition

↓

Cache

[text]
Expression Tree

↓

Presentation Model

↓

Cache

Only affected stages should recompute after behavioural changes.

Observability

Future runtime implementations should expose pipeline telemetry.

Examples include:

- execution time
- cache utilisation
- invalidation reasons
- behavioural triggers
- presentation updates

Observability exists to improve runtime quality.

Not to influence behavioural architecture.

Multi-Device Execution

Every device executes the same conceptual pipeline.

Desktop.

↓

Pipeline.

Phone.

↓

Pipeline.

Television.

↓

Pipeline.

Only Presentation differs.

Behavioural execution remains identical.

Plugins

Extensions contribute:

- behaviour
- information
- relationships

Plugins never participate directly in pipeline execution.

The Composition Engine determines:

- execution order
- stage boundaries
- presentation

Every extension therefore inherits identical runtime behaviour.

Good Examples

Playback

Behaviour.

↓

Runtime World.

↓

Timeline updates.

↓

Presentation updates.

↓

Rendering.

Only affected stages execute.

Reading

Chapter changes.

↓

Composition updates.

↓

Typography updates.

↓

Presentation.

Reading continues naturally.

Search

Search opens.

↓

Overlay Expressions.

↓

Adaptive Layout.

↓

Presentation.

The underlying World remains intact.

Anti-patterns

Full Pipeline

Executing every stage for every behavioural update.

Mutable Stages

Pipeline stages modifying upstream data.

Rendering Pipeline

Rendering engine influencing behavioural execution.

Platform Pipelines

Different clients inventing different runtime architectures.

Runtime Pipeline Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> RuntimeWorld
RuntimeWorld --> CompositionSolver
CompositionSolver --> ExpressionResolution
ExpressionResolution --> RuntimeHierarchy
RuntimeHierarchy --> AdaptiveLayout
AdaptiveLayout --> PresentationModel
PresentationModel --> Rendering
```

Every behavioural event travels through one deterministic architectural pipeline.

Relationship To Future Chapters

The next chapter defines **Composition Caching**.

Runtime Pipelines explain:

How runtime stages execute.

Composition Caching explains:

How the results of those stages are reused efficiently while preserving deterministic behaviour.

Together they define the execution model of the Composition Engine.

Summary

Runtime Pipelines transform behavioural events into presentation through one deterministic execution architecture.

Each stage performs one responsibility.

No stage duplicates another.

The runtime therefore remains:

- predictable,
- incremental,
- cacheable,
- platform independent.

Users should never perceive these pipelines.

They should simply experience a World that continually understands and responds to them.

Review Status

Status

Draft

Next File

09-composition-caching.md

Composition Caching

Purpose

The Composition Engine continuously solves the user's World.

Without caching, every behavioural update would require the entire runtime pipeline to execute repeatedly.

This would reduce performance while providing no additional understanding.

Composition Caching exists to preserve deterministic behaviour while minimising unnecessary recomputation.

The objective is not simply performance.

The objective is preserving continuity.

Definition

Within MDS, **Composition Caching** is defined as:

The deterministic reuse of previously solved runtime artefacts whenever behavioural understanding remains unchanged.

Composition Caching preserves:

- behavioural correctness,
- runtime efficiency,
- temporal continuity.

It should never alter the behaviour produced by the Composition Engine.

Philosophy

Caching should optimise work.

It should never optimise behaviour.

Users should experience:

[text]
Same Behaviour

↓

Same Composition

Whether the result was:

- recomputed,
- retrieved from cache,

should remain entirely invisible.

Behaviour Before Cache

Caching always follows behaviour.

Conceptually.

[text]
Behaviour

↓

Runtime World

↓

Cache Decision

↓

Pipeline

↓

Presentation

Behaviour always determines whether cached information remains valid.

The cache never determines behaviour.

Cacheable Artefacts

The Composition Engine may cache several conceptual artefacts.

Examples include:

[text]
Runtime World Snapshot

↓

Composition

↓

Expression Tree

↓

Presentation Model

↓

Adaptive Layout

Each stage may possess its own cache.

Stages remain independently invalidatable.

Runtime World Cache

Immutable Runtime World snapshots provide the foundation for deterministic caching.

Conceptually.

[text]
Runtime World

↓

Snapshot

↓

Cache Key

Future runtime systems may compare snapshots to determine whether downstream stages require recomputation.

Composition Cache

If:

- Behaviour,
- Focus,
- Context,
- Relationships

remain unchanged...

The solved Composition may be safely reused.

Behaviour remains the authoritative invalidation mechanism.

Expression Cache

Expression Trees may also be cached.

Example.

Playback progresses.

↓

Timeline Expression updates.

↓

Hero Expression unchanged.

Only affected Expressions require regeneration.

Incremental updates preserve continuity while improving runtime performance.

Layout Cache

Adaptive Layout should remain cacheable.

Examples.

Device unchanged.

↓

Layout reused.

Orientation changes.

↓

Layout invalidated.

Behaviour remains identical.

Only spatial presentation changes.

Presentation Cache

Presentation Models may also be cached.

Conceptually.

[text]
Expression Tree

↓

Presentation Model

↓

Cache

Presentation caching should remain independent from rendering technology.

Rendering engines may introduce additional caches.

These remain implementation concerns.

Behavioural Invalidation

Cache invalidation should always begin with behaviour.

Examples.

Playback progresses.

↓

Timeline invalidated.

Focus changes.

↓

Hero invalidated.

Search opens.

↓

Overlay invalidated.

Small behavioural changes should invalidate only the minimum required runtime artefacts.

Hierarchical Invalidation

Invalidation should propagate naturally through the runtime hierarchy.

Example.

[text]
Focus

↓

Hero

↓

Supporting Expressions

↓

Presentation

Peripheral Expressions should remain valid whenever possible.

Understanding should evolve incrementally.

Material Caching

Material behaviour should also participate.

Examples.

Runtime Atmosphere.

↓

Material Profiles.

↓

Refraction Fields.

↓

Cache.

Materials should reuse stable environmental information whenever practical.

Motion Caching

Motion Profiles may also be cached.

Examples.

Hero Transition.

↓

Resolved Motion Profile.

↓

Cache.

Only behavioural changes requiring different movement should invalidate Motion caches.

Deterministic Keys

Cache keys should derive only from deterministic runtime inputs.

Examples include:

- Runtime World Snapshot
- Behaviour
- Context
- Focus
- Accessibility
- Device Class

Implementation-specific values should never become behavioural cache keys.

Incremental Recomputation

Preferred.

[text]
Behaviour

↓

Small Runtime Change

↓

Local Recompute

Avoid.

[text]
Behaviour

↓

Entire Runtime Pipeline

↓

Complete Rebuild

Incremental recomputation preserves responsiveness while strengthening continuity.

Multi-Device Caching

Different devices may maintain independent Presentation caches.

However...

Composition caches should remain conceptually identical.

Desktop.

↓

Composition.

Phone.

↓

Same Composition.

Only Adaptive Layout differs.

Accessibility

Accessibility changes invalidate only the stages they affect.

Example.

Large Text.

↓

Typography.

↓

Presentation.

Composition remains unchanged.

Behaviour remains unchanged.

The cache should preserve unaffected runtime work.

Runtime Consistency

Cached artefacts must always produce identical runtime behaviour to recomputed artefacts.

Users should never perceive:

- stale hierarchy,
- delayed Expressions,
- inconsistent Materials.

Correctness always possesses higher priority than cache reuse.

Performance Profiles

Future runtime implementations may expose conceptual cache profiles.

Examples.

[text]

Performance

↓

Aggressive Cache

[text]
Balanced

↓

Incremental Cache

[text]
Development

↓

Minimal Cache

Profiles alter optimisation.

They never alter behaviour.

Plugins

Extensions contribute:

- behaviour,
- information,
- relationships.

Plugins never manage caches.

The Composition Engine owns:

- invalidation,
- recomputation,
- runtime reuse.

Every extension therefore inherits identical runtime performance characteristics.

Good Examples

Playback

Playback progresses.

↓

Timeline invalidated.

↓

Timeline recomputed.

↓

Everything else reused.

The runtime remains responsive.

Reading

Bookmark added.

↓

Bookmarks recomputed.

↓

Current Chapter remains unchanged.

↓

Hero reused.

Readers perceive uninterrupted continuity.

Search

Overlay appears.

↓

Overlay Expressions generated.

↓

Underlying Composition reused.

↓

Closing Search instantly restores previous Presentation.

Anti-patterns

Global Invalidation

Every behavioural event invalidates the entire runtime.

Behavioural Cache

Cache determines behavioural correctness.

Platform Cache

Rendering frameworks redefining runtime caching behaviour.

Stale World

Presentation continues using outdated Runtime World snapshots.

Composition Caching Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> RuntimeWorld
RuntimeWorld --> CacheValidation
CacheValidation -->|Hit| PresentationModel
CacheValidation -->|Miss| CompositionSolver
CompositionSolver
CompositionSolver --> ExpressionResolution
ExpressionResolution --> PresentationModel
PresentationModel
PresentationModel --> Rendering
```

Caching should optimise execution.

Never behaviour.

Relationship To Future Chapters

The next chapter defines **Multi-Device Composition**.

Composition Caching explains:

How runtime work is reused efficiently.

Multi-Device Composition explains:

How one solved World is simultaneously projected across multiple devices while preserving identical behavioural understanding.

Together they complete the runtime execution architecture of the Composition Engine.

Summary

Composition Caching preserves one of the most important architectural characteristics of Mosaic.

The interface should continuously evolve...

without continuously rebuilding itself.

Behaviour determines change.

Caching simply avoids solving the same understanding twice.

Users should therefore experience:

- responsiveness,
- continuity,
- consistency,

without ever realising a cache existed.

Review Status

Status

Draft

Next File

10-multi-device-composition.md

Multi-Device Composition

Purpose

The Composition Engine solves one Runtime World.

Users, however, may experience that World across many different devices.

Examples include:

- phone
- tablet
- desktop
- television
- foldable devices
- future spatial interfaces
- voice interfaces

This chapter defines how one solved Composition becomes many presentations without fragmenting behavioural understanding.

There should never be:

- Mobile Composition
- TV Composition
- Desktop Composition

There should only ever be:

One Composition.

Definition

Within MDS, **Multi-Device Composition** is defined as:

The deterministic projection of one solved Runtime Composition across multiple presentation environments while preserving behavioural understanding.

Presentation adapts.

Understanding does not.

Philosophy

Traditional applications frequently solve interfaces separately.

Examples.

Mobile UI

↓

Desktop UI

↓

TV UI

These experiences often diverge over time.

Mosaic intentionally solves once.

Runtime World

↓

Composition

↓

Expressions

↓

Device Projection

↓

Presentation

The World remains identical.

Only its physical expression changes.

One Runtime

Every client consumes the same Runtime World.

Desktop.

↓

Runtime World.

Phone.

↓

Same Runtime World.

Television.

↓

Same Runtime World.

No client owns independent behavioural state.

The Runtime World remains the single source of truth.

One Composition

Likewise.

Every client consumes the same solved Composition.

Examples.

Hero

Timeline

Relationships

Progress

Actions

These Expressions remain identical regardless of device.

The Adaptive Layout system determines presentation.

Device Projection

The Composition Engine projects solved Expressions into device-specific layouts.

Conceptually.

[text]
Composition

↓

Expression Tree

↓

Adaptive Layout

↓

Phone

[text]
Composition

↓

Expression Tree

↓

Adaptive Layout

↓

Desktop

Behaviour remains identical.

Device Classes

Future implementations may define conceptual device classes.

Examples.

[text]
Phone

Tablet

Desktop

Television

Foldable

Voice

Spatial

Each device class receives its own Presentation Model.

No device receives a different Composition.

Viewing Distance

Projection should account for viewing distance.

Television.

↓

Greater spacing.

↓

Larger typography.

↓

Expanded Hero.

Phone.

↓

Closer reading.

↓

Compact hierarchy.

The behavioural language remains unchanged.

Input Independence

Different devices expose different interaction methods.

Examples.

- touch
- mouse
- keyboard
- remote
- voice

- gesture

These affect interaction.

They do not affect Composition.

Behaviour remains independent from input technology.

Expression Stability

Expressions should remain recognisable across every platform.

Timeline.

↓

Timeline.

Hero.

↓

Hero.

Relationships.

↓

Relationships.

Only visual expression changes.

Conceptual identity remains constant.

Progressive Projection

Smaller devices should progressively disclose information.

Not remove it.

Incorrect.

Desktop

↓

Cast

↓

Phone

↓

Removed

Preferred.

Desktop

↓

Cast

↓

Phone

↓

Collapsed

↓

Expandable

Understanding remains available.

Presentation adapts.

Simultaneous Devices

Future Mosaic experiences may span multiple devices simultaneously.

Examples.

Television.

↓

Playback.

Phone.

↓

Companion controls.

Tablet.

↓

Metadata.

Each device receives a different Presentation Model.

All consume the same Runtime World.

Behaviour remains synchronised.

Runtime Synchronisation

Every device should observe identical behavioural evolution.

Example.

Playback pauses.

↓

Television updates.

↓

Phone updates.

↓

Tablet updates.

↓

Runtime World remains authoritative.

Presentation latency should never alter behavioural ordering.

Material Consistency

Different devices may implement Materials differently.

Desktop.

↓

Premium Acrylic.

Phone.

↓

Simplified Acrylic.

Television.

↓

Greater perceived depth.

The Material language remains recognisably Mosaic.

Only rendering fidelity changes.

Typography Consistency

Editorial hierarchy remains identical.

Examples.

Hero.

↓

Heading.

Supporting.

↓

Body.

Caption.

↓

Caption.

Different devices may alter:

- spacing
- measure
- scale

Editorial meaning remains unchanged.

Motion Consistency

Motion sequencing should remain identical.

Examples.

Hero moves.

↓

Supporting responds.

↓

Environment settles.

Different platforms may adjust:

- duration
- interpolation
- rendering fidelity

Behavioural ordering must remain identical.

Accessibility

Accessibility should remain device independent.

Large text.

↓

All devices.

Reduced motion.

↓

All devices.

High contrast.

↓

All devices.

Accessibility profiles belong to the Runtime World.

Not individual platforms.

Runtime Identity

Users should always feel they are interacting with:

The same Companion.

Changing device should never require relearning:

- hierarchy
- navigation
- behaviour
- editorial language

Only presentation changes.

Device Handover

Future implementations may support seamless device handover.

Example.

Watching.

↓

Television.

↓

Continue on Phone.

↓

Same Runtime World.

↓

Same Hero.

↓

Same Progress.

↓

Same Composition.

The transition should feel like continuing the same experience rather than opening a different application.

Plugins

Extensions contribute:

- behaviour
- information

- relationships

Plugins never target specific devices.

The Composition Engine determines how extension content appears everywhere.

Every extension therefore automatically supports future devices.

Good Examples

Television

Hero.



Large presentation.



Minimal controls.



Immersive experience.

Phone

Hero.



Compact presentation.



Progressive disclosure.



Comfortable touch interaction.

The behavioural language remains identical.

Foldable

One Runtime World.



Adaptive projection.



Expanded editorial layout.



Same Expressions.

The experience evolves naturally with the device.

Anti-patterns

Mobile Features

Mobile receives different behaviour.

Platform Hierarchy

Television invents different conceptual priorities.

Device Plugins

Extensions creating independent device interfaces.

Separate Runtime

Every client maintaining its own behavioural state.

Multi-Device Composition Model

[mermaid]
flowchart TD

```
RuntimeWorld
RuntimeWorld --> Composition
Composition --> ExpressionTree
ExpressionTree --> AdaptiveLayout
AdaptiveLayout --> Desktop
AdaptiveLayout --> Phone
AdaptiveLayout --> Tablet
AdaptiveLayout --> Television
AdaptiveLayout --> Voice
Desktop --> Presentation
Phone --> Presentation
Tablet --> Presentation
Television --> Presentation
Voice --> Presentation
```

One Runtime World.

One Composition.

Many Presentations.

Relationship To Future Chapters

The next chapter defines **Composition Engine Governance**.

Multi-Device Composition explains:

How one solved World becomes many device experiences.

Governance explains:

How the Composition Engine evolves while preserving one consistent behavioural architecture across every future client.

Together they complete the architectural foundation of the Mosaic runtime.

Summary

Multi-Device Composition ensures that Mosaic remains one platform rather than a collection of applications.

Users should experience:

- one World,
- one Companion,
- one behavioural language,

regardless of:

- screen size,
- input method,
- rendering technology,
- future devices.

Presentation may change.

Understanding never should.

Review Status

Status

Draft

Next File

11-governance.md

Composition Engine Governance

Purpose

The Composition Engine is the architectural heart of Mosaic.

Every runtime experience ultimately depends upon it.

Unlike rendering technologies, which may evolve frequently, the Composition Engine defines how the platform thinks.

Poor governance would gradually fragment:

- behavioural consistency,
- runtime hierarchy,
- composition,
- plugin integration,
- cross-platform parity.

This chapter defines how the Composition Engine should evolve while preserving one coherent runtime architecture.

Governance Philosophy

The Composition Engine should evolve technically.

Its behavioural model should remain remarkably stable.

The objective is not preserving implementation.

It is preserving:

- understanding,
- determinism,
- continuity,
- compositional integrity.

Future runtime technologies should strengthen these ideas rather than replacing them.

Runtime Is Architecture

The Composition Engine is not middleware.

It is architecture.

Changing:

- Runtime Hierarchy,
- Expression Resolution,

- Behaviour Orchestration,

changes how users understand Mosaic.

These changes therefore require architectural review.

Stable Responsibilities

The following concepts should remain highly stable.

- Runtime World
- Composition Solver
- Expression Resolution
- Runtime Hierarchy
- Behaviour Orchestration
- Adaptive Composition

These concepts define the behavioural identity of Mosaic.

Evolvable Responsibilities

The following may evolve continuously.

- caching strategies
- graph implementations
- execution pipelines
- scheduling
- runtime optimisation
- parallel execution

Implementation may evolve.

Behaviour should remain stable.

Engine Ownership

Composition Engine responsibilities are intentionally separated.

Layer	Owner
Runtime Philosophy	Design Systems
Composition Solver	Runtime Architecture
Expression Resolution	Runtime Architecture
Adaptive Layout	Runtime Architecture
Presentation Model	Client Runtime

Layer	Owner
Rendering	Client Platform

Ownership preserves conceptual integrity while allowing implementation to improve independently.

Introducing New Runtime Behaviour

Before introducing new runtime behaviour ask:

Question One

Does the Runtime World already communicate this?

Question Two

Does the Composition Solver already solve this?

Question Three

Could this be expressed through existing Expressions?

Question Four

Will this remain meaningful after future rendering technologies change?

Question Five

Would another contributor naturally discover this behaviour?

If uncertainty remains...

The proposal should continue evolving before implementation.

Runtime Drift

Runtime Drift occurs when:

- platforms solve behaviour differently,
- hierarchy diverges,
- plugins bypass composition,
- expressions lose semantic consistency,
- presentation begins influencing behaviour.

Runtime Drift weakens the entire Mosaic architecture.

It should therefore be treated as architectural debt.

Runtime Debt

Examples include:

- duplicated runtime logic,
- presentation-driven behaviour,
- component-owned hierarchy,
- plugin-owned composition,
- undocumented runtime exceptions.

Runtime Debt should be reduced continuously.

The Composition Engine should become simpler over time.

Not more complicated.

Solver Governance

The Composition Solver should always remain:

- deterministic,
- behavioural,
- presentation independent.

The Solver should never begin reasoning about:

- widgets,
- rendering,
- layouts,
- platforms.

Its only responsibility is understanding.

Expression Governance

Expressions represent one of the strongest architectural contracts within Mosaic.

New Expressions should be introduced sparingly.

Before creating a new Expression ask:

- Can an existing Expression communicate this?
- Is this actually new understanding?
- Is this merely another presentation?

The Expression vocabulary should remain intentionally compact.

Adaptive Layout Governance

Adaptive Layout should never redefine:

- hierarchy,
- behaviour,
- editorial structure.

Layout projects understanding.

It does not create understanding.

This distinction should remain inviolable.

Accessibility Governance

Accessibility always possesses higher authority than adaptive presentation.

No optimisation should weaken:

- hierarchy,
- continuity,
- understanding,
- behavioural clarity.

Accessibility modifies presentation.

Never runtime behaviour.

Plugin Governance

Extensions must never define:

- hierarchy,
- expressions,
- runtime pipelines,
- behavioural sequencing.

Plugins contribute:

- behaviour,
- information,
- relationships.

The Composition Engine owns runtime understanding.

This guarantees one coherent behavioural language throughout the ecosystem.

Review Questions

Every runtime proposal should answer:

- Does this strengthen understanding?
- Does this preserve determinism?
- Does this improve continuity?
- Does this reinforce behavioural hierarchy?
- Would users still recognise Mosaic after this change?
- Is this solving behaviour or implementation?

If the proposal exists primarily because a runtime optimisation is available...

It should be reconsidered.

Behaviour always possesses higher authority than optimisation.

Validation

Future tooling should automatically validate:

- deterministic solving
- Expression integrity
- hierarchy consistency
- behavioural sequencing
- cache correctness
- cross-platform parity

Validation should reinforce architectural review.

It should never replace behavioural reasoning.

Governance Workflow

[mermaid]
flowchart TD

```
Proposal
Proposal --> BehaviourReview
BehaviourReview --> ExistingConcept["ExistingConcept?"]
ExistingConcept["ExistingConcept?"] -->|Yes| Reuse
ExistingConcept["ExistingConcept?"] -->|No| ArchitectureReview
ArchitectureReview
ArchitectureReview --> RuntimeReview
RuntimeReview --> AccessibilityReview
AccessibilityReview --> Approval
Approval --> Implementation
```

Architectural refinement should always be preferred over expanding runtime concepts.

Success Criteria

The Composition Engine succeeds when:

- every client constructs the same understanding,
- runtime remains deterministic,
- behaviour always precedes presentation,
- extensions naturally integrate,
- contributors think in Worlds rather than interfaces,
- optimisation never weakens behavioural correctness.

Users should never perceive the Composition Engine.

They should simply experience a platform that always understands them.

Architectural Decisions

ADR	Decision
ADR-149	The Runtime World is the single behavioural source of truth.
ADR-150	The Composition Solver owns runtime understanding.
ADR-151	Expressions are the stable contract between runtime and presentation.
ADR-152	Adaptive Layout projects understanding without redefining it.
ADR-153	Extensions enrich the Runtime World but never own composition.

Review Status

Status

Draft

Next File

12-adr s .md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-006 preserve the architectural reasoning behind the Mosaic Composition Engine.

Every previous specification established:

- Vision
- Behaviour
- Design Language
- Runtime Systems

MDS-006 establishes how those systems become one continuously evolving runtime experience.

These ADRs explain why Mosaic treats runtime composition as a behavioural solving process rather than a rendering process.

Future contributors should understand these decisions before modifying the Composition Engine.

ADR Format

Every Mosaic ADR follows the standard structure.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR documents one architectural decision.

ADR-154

Title

Treat The Runtime World As The Primary Runtime Model

Status

Accepted

Context

Traditional applications frequently build interfaces from application state.

Founder workshops consistently favoured modelling the user's current World rather than interface state.

Decision

The Runtime World becomes the single behavioural source of truth.

Consequences

Every runtime subsystem consumes identical behavioural information.

Presentation becomes deterministic.

ADR-155

Title

Composition Is Solved Rather Than Authored

Status

Accepted

Context

Static interface templates struggle to communicate changing behavioural priorities.

Decision

The Composition Solver continuously constructs understanding from the Runtime World.

Consequences

The interface evolves naturally as behaviour changes.

ADR-156

Title

Introduce Expressions As A Runtime Abstraction

Status

Accepted

Context

Directly generating components tightly couples runtime behaviour to rendering frameworks.

Decision

Expressions become the stable contract between runtime understanding and presentation.

Consequences

Future clients remain implementation independent while sharing one conceptual runtime.

ADR-157

Title

Runtime Hierarchy Is Behavioural

Status

Accepted

Context

Traditional interfaces frequently derive hierarchy from layout.

Decision

Hierarchy is continuously solved from behaviour rather than geometry.

Consequences

Adaptive layouts preserve understanding across every device.

ADR-158

Title

Behaviour Orchestrates Every Runtime System

Status

Accepted

Context

Independent runtime subsystems frequently produce fragmented user experiences.

Decision

Every runtime subsystem evolves from one coordinated behavioural pipeline.

Consequences

Users experience one coherent World rather than multiple independent interface updates.

ADR-159

Title

Presentation Is A Runtime Product

Status

Accepted

Context

Traditional frameworks often treat presentation as the runtime model.

Decision

Presentation becomes the output of runtime solving.

Consequences

Rendering technologies remain replaceable without affecting runtime behaviour.

ADR-160

Title

Adaptive Layout Never Changes Behaviour

Status

Accepted

Context

Responsive layouts frequently alter behavioural meaning across devices.

Decision

Adaptive Layout projects solved understanding rather than redefining it.

Consequences

Every device communicates identical behavioural intent.

ADR-161

Title

Runtime Pipelines Must Remain Deterministic

Status

Accepted

Context

Predictable runtime behaviour is essential for replay, testing and synchronisation.

Decision

Every pipeline stage consumes immutable inputs and produces deterministic outputs.

Consequences

Caching, replay and multi-device synchronisation become reliable architectural capabilities.

ADR-162

Title

Extensions Enrich The Runtime World

Status

Accepted

Context

Allowing plugins to construct presentation fragments runtime consistency.

Decision

Extensions contribute behaviour, relationships and information only.

The Composition Engine owns runtime solving.

Consequences

Community extensions inherit every future runtime improvement automatically.

ADR Relationships

[mermaid]
flowchart TD

ADR154["Runtime World"]

ADR155["Composition Solver"]

ADR156["Expressions"]

ADR157["Runtime Hierarchy"]

ADR158["Behaviour Orchestration"]

ADR159["Presentation"]

ADR160["Adaptive Layout"]

ADR161["Deterministic Pipelines"]

ADR162["Extensions"]

ADR154 --> ADR155

ADR155 --> ADR156

ADR156 --> ADR157

ADR157 --> ADR160

ADR155 --> ADR158

ADR158 --> ADR161

ADR156 --> ADR159

ADR161 --> ADR162

Together these decisions establish the Composition Engine as a behavioural runtime architecture rather than a rendering framework.

Future ADRs

Future Composition Engine ADRs are expected to formalise:

- Predictive Composition
- AI-assisted Expression Selection
- Distributed Runtime Worlds
- Collaborative Runtime Sessions
- Spatial Composition
- Incremental Graph Solving
- Runtime Behaviour Personas
- Cloud-assisted Composition Pipelines

These intentionally remain outside the scope of MDS-006 Version 0.1.

ADR Governance

Composition Engine ADRs should change only when:

- behavioural research identifies deficiencies,

- runtime architecture fundamentally evolves,
- deterministic guarantees require refinement,
- the Mosaic Design Language itself changes.

Implementation technology alone should never justify architectural changes.

The runtime model should remain recognisably Mosaic regardless of future execution environments.

Summary

The ADRs contained within MDS-006 define the architectural heart of Mosaic.

Rather than constructing interfaces...

The Composition Engine constructs understanding.

Everything else:

- layouts,
- materials,
- typography,
- motion,
- rendering,

emerges naturally from that solved understanding.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Composition Engine is the architectural centre of Mosaic.

Every subsystem ultimately contributes to it.

Every client ultimately consumes it.

This guidance exists to help contributors think in terms of:

- Worlds,
- Behaviour,
- Composition,
- Understanding,

rather than:

- screens,
- widgets,
- layouts,
- pages.

When contributors naturally begin reasoning about Worlds instead of interfaces, the Composition Engine has achieved its purpose.

Think In Worlds

Never begin with:

"Which screen is this?"

Instead ask:

"What World is the user currently experiencing?"

Good.

[text]
watching

↓

Episode

↓

Playback

Poor.

[text]
Home Screen

↓

Episode Screen

↓

Player Screen

Screens are implementation.

Worlds are architecture.

Think In Behaviour

Every runtime change should begin with one question.

"What behaviour changed?"

Never begin with:

- button presses,
- navigation,
- layout,
- rendering.

Behaviour remains the highest authority.

Everything else follows naturally.

Solve Understanding

The Composition Engine exists to solve understanding.

Not layouts.

Not widgets.

Before implementing any runtime feature ask:

What should the user understand after this behaviour changes?

If the answer is unclear...

The implementation should probably not begin yet.

Expressions Before Components

Never think:

[text]
Card

↓

Button

↓

List

Instead think:

[text]
Hero

↓

Timeline

↓

Actions

↓

Relationships

Components are merely one possible implementation of Expressions.

The Expression vocabulary should always remain conceptually richer than the component vocabulary.

Preserve Hierarchy

Hierarchy should always emerge from:

- Behaviour,
- Composition,
- Relationships.

Never from:

- screen position,
- component type,
- visual styling.

Behaviour owns hierarchy.

Presentation communicates it.

Runtime Before Rendering

Rendering should remain the final concern.

Preferred.

[text]
Behaviour

↓

Composition

↓

Expressions

↓

Presentation

↓

Rendering

Avoid.

[text]
Widgets

↓

State

↓

Behaviour

The runtime should always lead implementation.

Respect Determinism

Every behavioural decision should produce deterministic runtime behaviour.

Given identical:

- Runtime World,
- Behaviour,
- Relationships,

the Composition Engine should produce identical Composition.

Predictability is significantly more valuable than cleverness.

Think Incrementally

Behaviour rarely changes everything.

Before recomputing ask:

What actually changed?

Preferred.

[text]
Timeline

↓

Updates

Avoid.

[text]
Entire World

↓

Rebuilt

Incremental evolution preserves both continuity and performance.

Respect Adaptive Layout

Adaptive Layout exists to project understanding.

Not redefine it.

Changing devices should never require changing:

- Expressions,
- Behaviour,
- Runtime Hierarchy.

Only presentation should adapt.

Plugins Enrich Worlds

Extensions should contribute:

- information,
- relationships,
- behaviour,
- capabilities.

Plugins should never attempt to solve:

- hierarchy,
- composition,
- layouts,
- runtime presentation.

The Composition Engine owns those responsibilities.

Accessibility

Accessibility modifies presentation.

It never modifies behaviour.

Examples.

Large text.

↓

Presentation changes.

Behaviour.

↓

Unchanged.

Reduced Motion.



Presentation changes.

Composition.



Unchanged.

Behavioural understanding should remain identical.

Platform Independence

Before implementing runtime behaviour ask:

Would this behave identically on:

- Web?
- Flutter?
- Television?
- Voice?
- Future platforms?

Presentation may differ.

Runtime behaviour should not.

Optimise Carefully

Performance optimisation should never alter behavioural correctness.

Preferred.

[text]
Same Runtime



Better Cache

Avoid.

[text]
Different Runtime



Faster Execution

Optimise execution.

Never understanding.

Common Mistakes

Avoid the following.

Screen Thinking

Beginning runtime architecture from interface pages.

Component Thinking

Allowing widgets to determine runtime behaviour.

Layout Thinking

Treating adaptive layout as runtime composition.

Platform Thinking

Building independent runtime models for different clients.

Plugin Ownership

Allowing extensions to bypass the Composition Engine.

Behaviour Duplication

Multiple runtime systems independently solving the same behaviour.

Composition Review Questions

Before implementing runtime logic ask:

- What World currently exists?
- What behaviour changed?
- What should the user understand next?
- Does this strengthen continuity?
- Could an existing Expression already communicate this?
- Would this still work on every Mosaic client?

If uncertainty remains...

Return to the Runtime World before implementation.

Composition Checklist

Every runtime implementation should satisfy the following.

- [] Runtime World remains authoritative.

- ☐ Behaviour initiated every runtime change.
- ☐ Composition Solver owns understanding.
- ☐ Expressions remain presentation independent.
- ☐ Adaptive Layout preserves behavioural meaning.
- ☐ Accessibility remains intact.
- ☐ Runtime behaviour remains deterministic.
- ☐ Platform independence is preserved.

Final Guidance

The Composition Engine should eventually disappear from conscious thought.

Contributors should stop asking:

"How should this interface work?"

and instinctively begin asking:

"How should the user's World evolve?"

When every contributor naturally thinks in Worlds rather than interfaces, Mosaic stops behaving like an application.

It begins behaving like a companion that always understands where the user is, what matters now, and how to guide them there without ever making them think about the machinery underneath.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the terminology introduced by **MDS-006 — Composition Engine**.

Unlike previous glossaries, these definitions describe the runtime architecture responsible for constructing the user's World.

These terms intentionally remain independent from:

- UI frameworks,
- rendering engines,
- platforms,
- programming languages.

Future specifications should reuse these definitions consistently.

A

Adaptive Layout

The runtime process that projects one solved Composition into different presentation environments.

Adaptive Layout changes:

- spatial organisation,
- density,
- presentation.

It never changes:

- behaviour,
- hierarchy,
- understanding.

B

Behaviour Orchestration

The coordinated execution of every runtime subsystem following behavioural change.

Behaviour Orchestration ensures:

- Composition,
- Materials,
- Typography,

- Motion,
- all evolve together.

C

Composition

The solved understanding of the user's current World.

Composition exists independently from:

- layouts,
- components,
- rendering.

Composition Solver

The deterministic runtime system responsible for transforming the Runtime World into a solved Composition.

The Solver determines:

- hierarchy,
- priority,
- grouping,
- Expressions.

It never determines rendering.

E

Expression

A runtime representation of one conceptual responsibility.

Examples include:

- Hero
- Timeline
- Progress
- Relationships
- Actions

Expressions are presentation independent.

Expression Resolution

The deterministic process that transforms solved Composition into reusable runtime Expressions.

Expression Resolution bridges runtime understanding and presentation.

H

Hierarchy Profile

A conceptual runtime artefact representing the resolved behavioural hierarchy of the current Composition.

Hierarchy Profiles improve:

- determinism,
- caching,
- incremental updates.

P

Presentation Model

The platform-independent runtime description consumed by rendering engines.

The Presentation Model contains:

- Expressions,
- Materials,
- Typography,
- Motion,
- Interaction metadata.

Components are generated afterwards.

R

Runtime World

The continuously evolving behavioural model representing the user's current reality.

The Runtime World is the single source of truth for:

- Focus,
- Context,
- Behaviour,
- Relationships.

Every runtime subsystem consumes it.

Runtime Pipeline

A deterministic execution stage within the Composition Engine.

Pipelines transform:

Behaviour

↓

Understanding

↓

Presentation

Each pipeline owns one architectural responsibility.

Runtime Snapshot

An immutable representation of the Runtime World used during one orchestration cycle.

Snapshots improve:

- replay,
- testing,
- caching,
- deterministic execution.

S

Solver

See **Composition Solver**.

Within Mosaic, "Solver" always refers to runtime understanding rather than layout computation.

W

World

The user's current behavioural reality.

A World contains:

- Focus,
- Context,
- Relationships,

- Behaviour,
- Information.

The Composition Engine continuously solves this World into understandable presentation.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion
MDL-002 Principles	Behaviour First
MDL-003 Mental Model	World, Focus
MDL-004 Interaction Model	Behaviour
MDL-005 Composition Model	Hero, Hierarchy
MDS-001 Design Token Architecture	Runtime Resolution
MDS-002 Colour System	Runtime Atmosphere
MDS-003 Material System	Material Resolution
MDS-004 Typography System	Editorial Hierarchy
MDS-005 Motion System	Behavioural Motion

Terminology Rules

Future contributors should:

- describe Worlds before screens,
- describe behaviour before rendering,
- describe Expressions before components,
- distinguish Composition from presentation,
- avoid platform-specific terminology inside architectural specifications.

Runtime terminology should remain independent from implementation technology.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences and conceptual foundations that informed **MDS-006 — Composition Engine**.

Unlike implementation documentation, these references explain *why* the Composition Engine exists rather than prescribing specific runtime technologies.

The Composition Engine intentionally combines ideas from:

- behavioural architecture,
- runtime systems,
- graph modelling,
- information architecture,
- reactive systems,
- deterministic execution,

into one coherent runtime capable of continuously constructing the user's World.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Design Token Architecture 3. Colour System 4. Material System 5. Typography System 6. Motion System 7. Composition Engine 8. Platform Runtime

The Mosaic Design Language remains the primary architectural authority.

External implementation technologies exist only to realise this model.

Internal References

MDL-001 — Vision

Provides:

- Companion philosophy
- Entertainment-first thinking
- Long-term product identity

The Composition Engine exists to continuously express the Companion rather than render isolated interfaces.

MDL-002 — Principles

Provides:

- Behaviour Before Interface
- Context Before Prediction
- Calm Interfaces
- Every Feature Earns Its Place

The Composition Engine continuously applies these principles at runtime.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Relationships
- Information

The Runtime World is the implementation of the Mental Model.

Everything within the Composition Engine begins here.

MDL-004 — Interaction Model

Provides:

- Behaviour
- Continuity
- Runtime evolution
- Interaction flow

Behaviour becomes the primary trigger for every runtime pipeline.

The Composition Engine never initiates behaviour.

It responds to it.

MDL-005 — Composition Model

Provides:

- Hero
- Hierarchy
- Priority
- Density
- Anchors
- Expressions

The Composition Solver is the runtime implementation of the Composition Model.

MDS-001 — Design Token Architecture

Provides:

- Runtime Resolution
- Semantic hierarchy
- Deterministic pipelines

The Composition Engine produces Presentation Models consumed by downstream Design System specifications.

MDS-002 — Colour System

Provides:

- Runtime Atmosphere
- Adaptive colour
- Semantic colour

The Composition Engine determines where Runtime Atmosphere should be expressed.

The Colour System determines how.

MDS-003 — Material System

Provides:

- Material Hierarchy
- Refraction
- Runtime Material Resolution

The Composition Engine assigns Material Intent.

The Material System resolves physical behaviour.

MDS-004 — Typography System

Provides:

- Editorial Hierarchy
- Runtime Typography
- Reading Rhythm

Expressions communicate editorial intent.

Typography communicates it to the user.

MDS-005 — Motion System

Provides:

- Behavioural Motion
- Runtime Motion Resolution
- Behaviour Orchestration

The Composition Engine supplies behavioural understanding.

The Motion System communicates its evolution.

Future Specifications

The following specifications directly depend upon MDS-006.

- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications implement presentation.

They never redefine runtime understanding.

Runtime Architecture

The Composition Engine intentionally follows a runtime-first architecture.

Conceptually.

[text]
Behaviour

↓

World

↓

Composition

↓

Presentation

↓

Rendering

Traditional interface frameworks often reverse this ordering.

Mosaic deliberately does not.

Behavioural Systems

One of the strongest influences behind the Composition Engine is behavioural architecture.

Rather than modelling:

- pages,
- routes,
- layouts,

the runtime models:

- behaviour,
- understanding,
- continuity,
- relationships.

Presentation naturally emerges afterwards.

Information Architecture

The Composition Engine treats information as the primary architectural resource.

Examples include:

- Hero
- Relationships
- Progress
- Context
- Behaviour

Information exists before interface.

The runtime simply determines how it should be understood.

Deterministic Execution

The Composition Engine intentionally assumes deterministic execution.

Identical Runtime Worlds should always produce identical:

- Composition
- Expressions
- Presentation Models

This assumption enables:

- replay,
- testing,
- caching,
- synchronisation,

- platform independence.

Runtime Graphs

Future implementations may internally represent the Runtime World using graph-based models.

The graph itself is intentionally **not** part of the architectural contract.

The architectural contract remains:

[text]
Runtime World

↓

Composition Solver

↓

Expressions

↓

Presentation

Graph technology is replaceable.

Behaviour is not.

Reactive Systems

The Composition Engine intentionally behaves as a reactive runtime.

Behaviour changes.

↓

Runtime World updates.

↓

Composition evolves.

↓

Presentation evolves.

This differs significantly from request-driven interface architectures.

Platform Independence

The Composition Engine intentionally separates:

[text]
Behaviour

↓

Composition

↓

Presentation

↓

Rendering

Every client therefore shares identical behavioural understanding while remaining free to implement presentation using:

- Flutter
- React
- SwiftUI
- Compose
- Future runtime technologies

without altering the architecture.

Mosaic-Specific Influences

The Composition Engine emerged directly from founder exploration.

Major architectural discoveries included:

- Interfaces should be solved rather than authored.
- Behaviour should own hierarchy.
- Expressions should separate understanding from components.
- Runtime should model Worlds rather than screens.
- Presentation should remain an implementation detail.

These discoveries collectively define the architectural identity of Mosaic.

Relationship To The Companion

The Composition Engine represents the reasoning layer of the Companion.

Conceptually.

[text]
World

↓

Behaviour

↓

Understanding

↓

Composition

↓

Presentation

↓

User

The Companion understands first.

The interface communicates second.

That ordering should remain fundamental throughout the lifetime of Mosaic.

Normative References

Required reading before contributing to MDS-006.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System

Together these specifications define the conceptual foundation of the Composition Engine.

Informative References

Future contributors may also wish to review:

- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications describe how Expressions become reusable presentation primitives while preserving the runtime architecture established here.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- runtime behaviour,
- behavioural architecture,
- composition solving,

- implementation boundaries.

The objective is to preserve architectural reasoning rather than catalogue runtime technologies.

Completion

This concludes **MDS-006 — Composition Engine**.

The next specification in the Mosaic Design System is:

MDS-007 — Tile Framework

Where MDS-006 defines **how the user's World is solved**, MDS-007 defines **how that solved understanding becomes reusable presentation primitives**.

It formalises:

- Tile philosophy
- Expression-to-Tile mapping
- Tile lifecycle
- Adaptive Tile behaviour
- Tile composition
- Tile interaction
- Tile orchestration
- Runtime tile resolution
- Extension tile integration

The Tile Framework bridges the final gap between the runtime intelligence of the Composition Engine and the visual Component Library, allowing Mosaic to construct interfaces that remain adaptive, behavioural and entirely driven by understanding rather than static layouts.